(19)国家知识产权局

(12)发明专利申请

(10)申请公布号 CN 122472020 A
(43)申请公布日 2026.07.28

(21)申请号 202610589244.0

(22)申请日 2026.04.30

(71)申请人 苏州数猎科技有限公司
地址 215104 江苏省苏州市吴中区越溪街道塔韵商务广场1711号

(72)发明人 程凯征

(74)专利代理机构 北京盛询知识产权代理有限公司 11901
专利代理师 蔺巍

(51)Int.Cl.
G06F 40/205(2020.01)
G06F 40/211(2020.01)
G06F 40/30(2020.01)
G06F 40/166(2020.01)
G06N 5/04(2023.01)
G06N 5/022(2023.01)

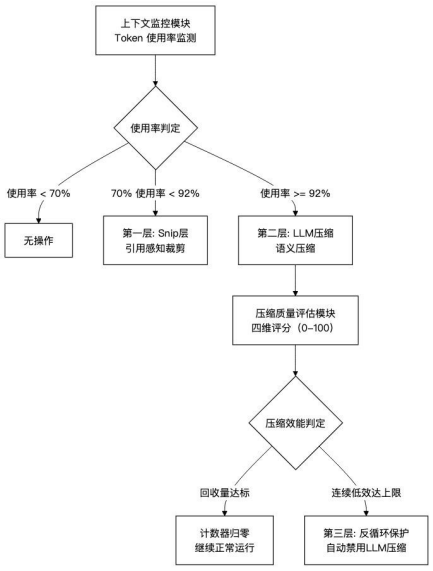
权利要求书2页 说明书11页 附图3页

(54)发明名称

一种面向大语言模型智能体的多层级自适应上下文压缩方法及系统

(57)摘要

本发明公开了一种面向大语言模型智能体的多层级自适应上下文压缩方法及系统,方法包括:持续监测当前智能体对话上下文的Token使用率;当Token使用率超过第一预设阈值时,则执行第一层压缩处理,其中,第一压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容;当Token使用率超过第二预设阈值时,执行第二层压缩处理,将新构建的上下文架构替换原有的历史对话;其中,第二预设阈值大于第一预设阈值;监测每次第二层压缩处理执行所产生的Token回收量,当连续N次压缩的Token回收量均低于最小回收阈值时,则自动禁用第二层压缩。本发明有效解决了现有技术中均匀压缩导致近期重要信息被过度压缩的问题。



CN 122472020 A

1.一种面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,包括:

持续监测当前智能体对话上下文的Token使用率;

当所述Token使用率超过第一预设阈值时,则执行第一层压缩处理,其中,所述第一层压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容;

当所述Token使用率超过第二预设阈值时,执行第二层压缩处理,将新构建的上下文架构替换原有的历史对话;其中,所述第二预设阈值大于所述第一预设阈值;

监测每次第二层压缩处理执行所产生的Token回收量,当连续N次压缩的Token回收量均低于最小回收阈值时,则自动禁用所述第二层压缩。

2.根据权利要求1所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,监测所述当前智能体对话上下文的Token使用率,包括:

基于文本中的字符类型分布特征进行Token数量估算,其中,对CJK字符占比超过预设比例的第一类文本采用第一字符/Token比率估算,对ASCII字符主导的第二类文本采用第二字符/Token比率估算,其中,第一比率值低于第二比率值。

3.根据权利要求1所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,执行所述第一层压缩处理,包括:

划定包含最近K条消息的保护区,其中K为正整数;

将保护区内所有消息的文本内容合并为参考文本集,提取保护区外每条工具结果消息中的特征关键词;

将所述特征关键词与所述参考文本集进行匹配,判断所述特征关键词是否存在于所述保护区内的消息文本中;

若判定所述特征关键词未被引用,则对相应工具结果消息执行裁剪,保留剩余部分内容并标记裁剪信息。

4.根据权利要求3所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,所述特征关键词包括文件路径特征和代码标识符特征。

5.根据权利要求1所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,执行所述第二层压缩处理,包括:

基于当前对话的复杂度特征,确定需保留的最近对话轮数;

根据所述最近对话轮数确定安全分割点,将消息序列划分为保留区和待压缩区,其中,所述安全分割点的位置为确保工具调用指令及对应的工具结果消息不被分割于不同区域;

对所述待压缩区内的消息按时间顺序划分若干年龄层,并设置与各年龄层对应的差异化压缩强度;

调用语言模型依据所述差异化压缩强度生成结构化摘要;

基于所述保留区的原始消息和所述结构化摘要重建上下文。

6.根据权利要求5所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,基于当前对话的复杂度特征,确定需保留的最近对话轮数,包括:

分别评估消息长度维度、工具使用维度、异常指标维度以及代码密度维度的特征分值;

对每个维度的特征分值进行求和计算,获得复杂度总分;

基于基础保留轮数与所述复杂度总分的正相关关系,计算最优保留轮数,其中,所述基础保留轮数为预设默认值。

7.根据权利要求5所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,根据所述最近对话轮数确定安全分割点,包括:

根据所述需保留的最近对话轮数计算初始分割位置;

从所述初始分割位置沿消息序列向前遍历,若当前位置对应的消息角色为工具结果类型,则继续向前移动至非工具结果类型消息所在位置,并将相应位置确定为所述安全分割点。

8.根据权利要求5所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,对所述待压缩区内的消息按时间顺序划分若干年龄层,并设置与各年龄层对应的差异化压缩强度,包括:

根据消息进入待压缩区的时间先后和预置数量占比,将待压缩区内的消息划分为远古层、历史层和近期层,其中,所述远古层为最早进入待压缩区且在待压缩消息总量占比为A%的消息组成,所述历史层为在所述远古层最后一条消息后进入所述待压缩区且在待压缩消息总量占比为B%的消息组成,所述近期层为在所述历史层最后一条消息后进入所述待压缩区且在待压缩消息总量占比为C%的消息组成;

对于所述远古层,配置第一压缩强度,保留结论性描述;对于所述历史层,配置第二压缩强度,保留决策记录;对于所述近期层,配置第三压缩强度,保留技术细节;

其中,A+B+C=100,且所述第一压缩强度高于第二压缩强度,所述第二压缩强度高于第三压缩强度。

9.根据权利要求1所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,在所述第二层压缩处理执行后,还包括对压缩结果进行质量评估,包括:

基于摘要的结构完整性、信息密度、压缩效率及保留准确性四个维度计算加权综合质量分。

10.一种面向大语言模型智能体的多层级自适应上下文压缩系统,用于实现权利要求1-9任一项所述的面向大语言模型智能体的多层级自适应上下文压缩方法,其特征在于,包括:

上下文监测模块,用于持续监测当前智能体对话上下文的Token使用率;

第一层压缩模块,用于当所述Token使用率超过第一预设阈值时,则执行第一层压缩处理,所述第一层压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容;

第二层压缩模块,用于当所述Token使用率超过第二预设阈值时,执行第二层压缩处理,将新构建的上下文架构替换原有的历史对话;其中,所述第二预设阈值大于所述第一预设阈值;

反循环保护模块,用于监测每次第二层压缩处理执行所产生的Token回收量,当连续N次压缩的Token回收量均低于最小回收阈值时,则自动禁用所述第二层压缩。

一种面向大语言模型智能体的多层级自适应上下文压缩方法及系统

技术领域

[0001] 本发明涉及人工智能与自然语言处理技术领域,尤其涉及一种面向大语言模型智能体的多层级自适应上下文压缩方法及系统。

背景技术

[0002] 大语言模型(如GPT、Claude、DeepSeek等)在实际应用中受限于上下文窗口(Context Window)的最大Token容量。当智能体执行复杂任务时,对话历史、工具调用结果、代码片段等内容不断累积,极易超出上下文窗口限制,导致:

(1)信息截断:简单丢弃早期对话,造成任务上下文断裂,智能体"失忆";

(2)推理质量下降:过长的上下文导致模型注意力分散,关键信息被噪声淹没;

(3)成本浪费:大量无效的历史Token占据窗口,增加API调用费用;

(4)任务失败:超出窗口限制时被强制截断,正在进行的复杂任务被迫中断。

[0003] 目前业界主要采用以下方案处理上下文溢出问题,但均存在明显缺陷:

(1)固定长度截断法:按照先进先出(FIFO)原则,直接丢弃最早的消息。该方法简单粗暴,无法区分消息的重要性,可能丢失关键的任务目标和技术决策信息。

[0004] (2)固定摘要法:在达到阈值时将所有历史消息通过LLM生成一段固定格式的摘要。该方法缺乏对不同"年龄"消息的差异化处理,对所有历史信息施加相同的压缩力度,导致近期重要细节被过度压缩,而远期无关信息占据过多空间。

[0005] (3)向量检索法:将历史消息向量化存储于向量数据库,根据当前查询进行相似性检索。该方法引入了外部依赖和延迟,且难以捕获对话中的时序关系和因果链条,不适合需要完整对话上下文的智能体场景。

[0006] (4)单一阈值触发法:仅设置一个压缩阈值(如Token使用率达到90%时触发),缺乏渐进式预防机制。当对话中包含大量工具调用结果时,可能在短时间内从安全区间跳跃到临界状态,来不及执行有效压缩。

[0007] 因此,在大语言模型智能体的长对话场景中,如何在不丢失关键信息的前提下,高效、安全、自适应地管理和压缩对话上下文,使智能体在有限的上下文窗口内保持完整的任务理解能力和推理质量成为当前研究的热点问题。

发明内容

[0008] 为解决上述现有技术中的技术问题,本发明提出了一种面向大语言模型智能体的多层级自适应上下文压缩方法及系统,实现上下文的高效压缩管理。

[0009] 一方面,为实现上述目的,本发明提供了一种面向大语言模型智能体的多层级自适应上下文压缩方法,包括:

持续监测当前智能体对话上下文的Token使用率;

当所述Token使用率超过第一预设阈值时,则执行第一层压缩处理,其中,所述第

一层压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容；

当所述Token使用率超过第二预设阈值时，执行第二层压缩处理，将新构建的上下文架构替换原有的历史对话；其中，所述第二预设阈值大于所述第一预设阈值；

监测每次第二层压缩处理执行所产生的Token回收量，当连续N次压缩的Token回收量均低于最小回收阈值时，则自动禁用所述第二层压缩。

[0010] 优选地，监测所述当前智能体对话上下文的Token使用率，包括：

基于文本中的字符类型分布特征进行Token数量估算，其中，对CJK字符占比超过预设比例的第一类文本采用第一字符/Token比率估算，对ASCII字符主导的第二类文本采用第二字符/Token比率估算，其中，第一比率值低于第二比率值。

[0011] 优选地，执行所述第一层压缩处理，包括：

划定包含最近K条消息的保护区，其中K为正整数；

将保护区内所有消息的文本内容合并为参考文本集，提取保护区外每条工具结果消息中的特征关键词；

将所述特征关键词与所述参考文本集进行匹配，判断所述特征关键词是否存在于所述保护区内的消息文本中；

若判定所述特征关键词未被引用，则对相应工具结果消息执行裁剪，保留剩余部分内容并标记裁剪信息。

[0012] 优选地，所述特征关键词包括文件路径特征和代码标识符特征。

[0013] 优选地，执行所述第二层压缩处理，包括：

基于当前对话的复杂度特征，确定需保留的最近对话轮数；

根据所述最近对话轮数确定安全分割点，将消息序列划分为保留区和待压缩区，其中，所述安全分割点的位置为确保工具调用指令及对应的工具结果消息不被分割于不同区域；

对所述待压缩区内的消息按时间顺序划分若干年龄层，并设置与各年龄层对应的差异化压缩强度；

调用语言模型依据所述差异化压缩强度生成结构化摘要；

基于所述保留区的原始消息和所述结构化摘要重建上下文。

[0014] 优选地，基于当前对话的复杂度特征，确定需保留的最近对话轮数，包括：

分别评估消息长度维度、工具使用维度、异常指标维度以及代码密度维度的特征分值；

对每个维度的特征分值进行求和计算，获得复杂度总分；

基于基础保留轮数与所述复杂度总分的正相关关系，计算最优保留轮数，其中，所述基础保留轮数为预设默认值。

[0015] 优选地，根据所述最近对话轮数确定安全分割点，包括：

根据所述需保留的最近对话轮数计算初始分割位置；

从所述初始分割位置沿消息序列向前遍历，若当前位置对应的消息角色为工具结果类型，则继续向前移动至非工具结果类型消息所在位置，并将相应位置确定为所述安全分割点。

[0016] 优选地，对所述待压缩区内的消息按时间顺序划分若干年龄层，并设置与各年龄

层对应的差异化压缩强度,包括:

根据消息进入待压缩区的时间先后和预置数量占比,将待压缩区内的消息划分为远古层、历史层和近期层,其中,所述远古层为最早进入待压缩区且在待压缩消息总量占比为A%的消息组成,所述历史层为在所述远古层最后一条消息后进入所述待压缩区且在待压缩消息总量占比为B%的消息组成,所述近期层为在所述历史层最后一条消息后进入所述待压缩区且在待压缩消息总量占比为C%的消息组成;

对于所述远古层,配置第一压缩强度,保留结论性描述;对于所述历史层,配置第二压缩强度,保留决策记录;对于所述近期层,配置第三压缩强度,保留技术细节;

其中,A+B+C=100,且所述第一压缩强度高于第二压缩强度,所述第二压缩强度高于第三压缩强度。

[0017] 优选地,在所述第二层压缩处理执行后,还包括对压缩结果进行质量评估,包括:

基于摘要的结构完整性、信息密度、压缩效率及保留准确性四个维度计算加权综合质量分。

[0018] 另一方面,为实现上述目的,本发明还提供一种面向大语言模型智能体的多层级自适应上下文压缩系统,包括:

上下文监测模块,用于持续监测当前智能体对话上下文的Token使用率;

第一层压缩模块,用于当所述Token使用率超过第一预设阈值时,则执行第一层压缩处理,所述第一层压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容;

第二层压缩模块,用于当所述Token使用率超过第二预设阈值时,执行第二层压缩处理,将新构建的上下文架构替换原有的历史对话;其中,所述第二预设阈值大于所述第一预设阈值;

反循环保护模块,用于监测每次第二层压缩处理执行所产生的Token回收量,当连续N次压缩的Token回收量均低于最小回收阈值时,则自动禁用所述第二层压缩。

[0019] 与现有技术相比,本发明具有如下优点和技术效果:

(1)本发明通过三层递进的压缩架构(Snip层→LLM语义压缩层→反循环保护层),实现了上下文的高效压缩管理。在典型应用场景下,本发明达到了30%-60%的压缩比,即100K Token的对话上下文可被压缩至30K-60K Token,同时保持了任务的连续性和对话的连贯性。

[0020] (2)本发明通过将待压缩消息按时间远近划分为远古层、历史层和近期层,并对各层施加差异化的压缩强度,实现了“越旧越压、越新越保”的智能信息保留。通过该技术手段,近期工作细节的保留率接近100%,远期关键决策的保留率超过90%,有效解决了现有技术中均匀压缩导致近期重要信息被过度压缩的问题。

[0021] (3)本发明通过Snip层的轻量级预防性裁剪,平均减少了20%的深度压缩触发次数,降低了LLM API调用成本。同时,反循环保护机制通过监测压缩效能并在低效时自动禁用LLM压缩层,避免了约15%-30%的无效LLM调用,进一步降低了系统运行成本。

[0022] (4)本发明通过安全分割点算法,在压缩分割时从初始分割位置向前遍历跳过TOOL类型消息,确保工具调用指令与其执行结果始终处于同一区域,达到了压缩后上下文中不会出现“孤儿”工具结果的效果,保障了智能体推理过程不受干扰。

[0023] (5)本发明通过基于消息长度、工具使用频率、异常指示和代码密度四个维度的复

杂度评估算法,实现了保留轮数的动态自适应调整。在简单对话场景下自动保留较少轮数(如3轮)以最大化释放空间,在复杂调试场景下自动保留较多轮数(如10轮)以确保上下文完整性,无需人工干预即可自动适应不同对话模式。

[0024] (6)本发明通过结构完整性、信息密度、压缩效率和保留准确性四个维度的加权评分机制,实现了对每次压缩质量的量化评估。该机制使压缩过程具备了可观测性,能够及时发现压缩质量异常并触发告警,为压缩策略的持续优化提供了数据支撑。

附图说明

[0025] 构成本申请的一部分的附图用来提供对本申请的进一步理解,本申请的示意性实施例及其说明用于解释本申请,并不构成对本申请的不当限定。在附图中:

图1为本发明实施例的一种面向大语言模型智能体的多层级自适应上下文压缩方法流程图;

图2为本发明实施例的年龄感知差异化压缩示意图;

图3为本发明实施例的自适应保留轮数计算流程图;

图4为本发明实施例的安全分割点计算示意图。

具体实施方式

[0026] 需要说明的是,在不冲突的情况下,本申请中的实施例及实施例中的特征可以相互组合。下面将参考附图并结合实施例来详细说明本申请。

[0027] 需要说明的是,在附图的流程图示出的步骤可以在诸如一组计算机可执行指令的计算机系统中执行,并且,虽然在流程图中示出了逻辑顺序,但是在某些情况下,可以以不同于此处的顺序执行所示出或描述的步骤。

[0028] 本实施例提出了一种面向大语言模型智能体的多层级自适应上下文压缩方法,如图1,包括:

持续监测当前智能体对话上下文的Token使用率;

当所述Token使用率超过第一预设阈值时,则执行第一层压缩处理,其中,所述第一层压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容;

当所述Token使用率超过第二预设阈值时,执行第二层压缩处理,将新构建的上下文架构替换原有的历史对话;其中,所述第二预设阈值大于所述第一预设阈值;

监测每次第二层压缩处理执行所产生的Token回收量,当连续N次压缩的Token回收量均低于最小回收阈值时,则自动禁用所述第二层压缩。

[0029] 进一步地,监测所述当前智能体对话上下文的Token使用率,包括:

基于文本中的字符类型分布特征进行Token数量估算,其中,对CJK字符占比超过预设比例的第一类文本采用第一字符/Token比率估算,对ASCII字符主导的第二类文本采用第二字符/Token比率估算,其中,第一比率值低于第二比率值。

[0030] 具体地,在智能体主循环的每次迭代中,持续计算当前对话上下文的Token使用率。为避免依赖外部tokenizer库引入的额外开销和外部依赖,本实施例采用基于字符特征的快速Token估算方法,具体包括:

首先检测文本中CJK字符(中日韩统一表意文字)的占比,对于CJK字符密集文本

(占比超过30%),按约1.5字符/Token的比率估算;对于ASCII字符主导文本,按约4字符/Token的比率估算;混合文本则按加权比例计算。

[0031] 该方法精度约为±10%,但由于压缩阈值之间预留了充分的缓冲空间(如T1与T2之间约有22%的缓冲区间),该精度足以满足压缩决策需求,同时将计算耗时控制在1ms以内。

[0032] 基于Token使用率的监测结果,判定是否需要触发压缩以及压缩的紧急程度。本实施例设置三个压缩层级的触发条件:当使用率低于第一预设阈值T1(如70%)时,无需任何压缩操作;当使用率达到T1但低于第二预设阈值T2(如92%)时,触发第一层Snip层压缩;当使用率达到T2时,触发第二层LLM语义压缩。此外,当使用率达到100%时判定为紧急压缩,需立即执行第二层压缩;当使用率超过110%时判定为严重超限,需执行应急恢复。

[0033] 进一步地,执行所述第一层压缩处理,包括:

划定包含最近K条消息的保护区,其中K为正整数;

将保护区内所有消息的文本内容合并为参考文本集,提取保护区外每条工具结果消息中的特征关键词;

将所述特征关键词与所述参考文本集进行匹配,判断所述特征关键词是否存在于所述保护区内的消息文本中;

若判定所述特征关键词未被引用,则对相应工具结果消息执行裁剪,保留剩余部分内容并标记裁剪信息。

[0034] 具体地,当上下文Token使用率超过第一预设阈值T1(如70%)时,触发本步骤执行。本步骤的核心思想是:在达到深度压缩阈值之前,先主动清理对话中已经“过时”的工具调用结果,为后续对话腾出空间。工具调用结果(如文件内容读取、代码执行输出、搜索结果等)通常占据大量Token,但随着对话推进,其中大部分已不再被后续消息引用。

[0035] 具体执行流程如下:

首先,进行保护区划定。将最近K条消息(如K=6)标记为保护区,保护区内的消息不做任何裁剪,以确保当前正在进行的对话线索不受影响。

[0036] 其次,进行引用检测。将保护区内所有消息的文本内容合并为参考文本集。对保护区外的每条工具结果消息,从其内容的前N个字符(如N=500)中提取特征关键词。特征关键词包括两类:一类是文件路径特征(如 /src/main.py、config.json),另一类是代码标识符特征(如函数名calculate_score、类名DataProcessor)。将提取到的特征关键词与参考文本集进行匹配,判定该工具结果是否仍被当前对话所引用。

[0037] 然后,进行裁剪判定。裁剪判定遵循以下规则:若工具结果长度不超过最小阈值(如200字符),跳过不处理,因为短小的结果裁剪收益有限;若无法从工具结果中提取到特征关键词,采用保守策略保留原文不做裁剪,避免误删重要内容;若特征关键词在参考文本集中均未出现,判定该工具结果为“已过时”,执行裁剪。

[0038] 裁剪方式为保留工具结果的第一行(不超过100字符)作为索引摘要,其余内容替换为标记文本[... snipped, X chars ...],其中X为被裁剪的字符数。

[0039] 最后,进行Token重新计算。裁剪完成后,使用Token估算方法重新计算上下文总Token数和使用率。若使用率已降至T1以下,则裁剪完成;若使用率仍高于T2,则继续触发第二层压缩。

[0040] 本步骤在不影响当前对话连贯性的前提下,通常可节省10%-30%的Token占用,有

效延缓深度压缩的触发频率。

[0041] 进一步地,执行所述第二层压缩处理,包括:

基于当前对话的复杂度特征,确定需保留的最近对话轮数;

根据所述最近对话轮数确定安全分割点,将消息序列划分为保留区和待压缩区,其中,所述安全分割点的位置为确保工具调用指令及对应的工具结果消息不被分割于不同区域;

对所述待压缩区内的消息按时间顺序划分若干年龄层,并设置与各年龄层对应的差异化压缩强度;

调用语言模型依据所述差异化压缩强度生成结构化摘要;

基于所述保留区的原始消息和所述结构化摘要重建上下文。

[0042] 具体地,当上下文Token使用率超过第二预设阈值T2(如 92%)时,触发第二层LLM语义压缩。第二层压缩的第一步是根据多维复杂度动态计算应当完整保留的最近对话轮数,而非采用固定的保留轮数,以适应不同类型和复杂度的对话场景。

[0043] 进一步地,如图3,基于当前对话的复杂度特征,确定需保留的最近对话轮数,包括:

分别评估消息长度维度、工具使用维度、异常指标维度以及代码密度维度的特征分值;

对每个维度的特征分值进行求和计算,获得复杂度总分;

基于基础保留轮数与所述复杂度总分的正相关关系,计算最优保留轮数,其中,所述基础保留轮数为预设默认值。

[0044] 具体方法为:分析最近M条消息(如M=20),从以下四个维度评估当前对话的复杂度。

[0045] 第一个维度是消息长度维度,评估最近M条消息的平均字符数。若平均字符数大于2000字符,则该维度得2分;若大于1000字符但不超过2000字符,则得1分;其余情况得0分。较长的消息通常意味着更复杂的技术讨论,需要保留更多上下文。

[0046] 第二个维度是工具使用维度,统计最近M条消息中工具调用的总次数。若调用次数大于10次,则该维度得2分;若大于5次但不超过10次,则得1分;其余情况得0分。频繁的工具调用表明智能体正在执行复杂的多步骤操作,需要保留操作上下文。

[0047] 第三个维度是异常指示维度,检测最近M条消息中是否包含错误或异常相关的关键词(如error、exception、failed、traceback等)。若包含此类关键词,则该维度得1分;否则得0分。异常信息的存在表明智能体正处于问题排查阶段,需要保留错误上下文和修复尝试。

[0048] 第四个维度是代码密度维度,计算最近M条消息中包含代码特征(如代码块标记、缩进结构、编程语言关键字等)的消息所占比例。若比例大于50%,则该维度得2分;若大于25%但不超过50%,则得1分;其余情况得0分。高代码密度表明当前对话以编程任务为主,代码上下文的完整性至关重要。

[0049] 四个维度的评分之和构成复杂度总分,其取值范围为0至7分。最优保留轮数通过以下公式计算:

最优保留轮数=基础保留轮数+⌊复杂度总分÷2⌋;

其中基础保留轮数为预设的默认值(如5),⌊⌋表示向下取整运算。

[0050] 计算结果被约束在[Rmin, Rmax]范围内(如[3,10]),其中Rmin为最小保留轮数,Rmax为最大保留轮数。

[0051] 该算法的自适应效果为:在简单问答场景(总分0-1)下,保留3-5轮,最大化压缩空间释放;在代码调试场景(总分4-5)下,保留7-8轮,确保错误上下文和修复过程的完整;在复杂重构场景(总分6-7)下,保留9-10轮,保留完整的技术决策链。

[0052] 进一步地,根据所述最近对话轮数确定安全分割点,包括:

根据所述需保留的最近对话轮数计算初始分割位置;

从所述初始分割位置沿消息序列向前遍历,若当前位置对应的消息角色为工具结果类型,则继续向前移动至非工具结果类型消息所在位置,并将相应位置确定为所述安全分割点。

[0053] 具体地,确定保留轮数后,需要在消息序列中计算安全的分割点,将消息序列分为“待压缩区”和“保留区”两部分。

[0054] 安全分割点计算的核心约束是保护工具调用与工具结果的配对完整性。在LLM智能体对话中,一条ASSISTANT消息可能包含若干工具调用指令(tool_calls),紧随其后的若干TOOL消息是对应的执行结果。若分割点落在工具调用指令与其结果之间,会导致两个问题:其一,保留区中出现“孤儿”工具结果,即缺少父级调用指令的工具结果消息,LLM无法理解其来源和上下文;其二,压缩区中出现未闭合的工具调用指令,LLM在对这些消息进行摘要时产生困惑。

[0055] 如图4,安全分割算法的具体步骤为:首先,计算初始分割点P=消息总数-保留消息数。然后,从P位置向前遍历消息序列,检查P位置处消息的角色类型,若该消息角色为TOOL类型,则将P向前移动一位(即P=P-1),继续检查,直至P位置处的消息不是TOOL类型。

[0056] 设置安全底线:P不小于1,即至少保留1条消息用于压缩。最终返回P作为安全分割索引,P之前(不含P)的消息进入待压缩区,P及其之后的消息进入保留区。

[0057] 通过该算法,可确保工具调用指令与其所有执行结果始终处于同一区域(要么同时被压缩,要么同时被保留),不会被分割到不同区域。

[0058] 进一步地,对所述待压缩区内的消息按时间顺序划分若干年龄层,并设置与各年龄层对应的差异化压缩强度,包括:

对所述待压缩区内的消息按时间顺序划分若干年龄层,并设置与各年龄层对应的差异化压缩强度,包括:

根据消息进入待压缩区的时间先后和预置数量占比,将待压缩区内的消息划分为远古层、历史层和近期层,其中,所述远古层为最早进入待压缩区且在待压缩消息总量占比为A%的消息组成,所述历史层为在所述远古层最后一条消息后进入所述待压缩区且在待压缩消息总量占比为B%的消息组成,所述近期层为在所述历史层最后一条消息后进入所述待压缩区且在待压缩消息总量占比为C%的消息组成;

对于所述远古层,配置第一压缩强度,保留结论性描述;对于所述历史层,配置第二压缩强度,保留决策记录;对于所述近期层,配置第三压缩强度,保留技术细节;

其中,A+B+C=100,且所述第一压缩强度高于第二压缩强度,所述第二压缩强度高于第三压缩强度。

[0059] 具体地,如图2,确定安全分割点后,将待压缩区的消息按时间顺序划分为三个年龄层,对每个年龄层分配不同的压缩强度,实现“越旧越压、越新越保”的渐进式信息保留策略。

[0060] 第一个年龄层为远古层,对应待压缩区中最早的消息,占待压缩消息总量的前20%。对远古层施加高压缩强度,仅保留关键决策和里程碑结论,忽略过程细节和代码片段。例如,远古层中的多轮调试对话仅被保留为“经排查确认问题根因为X,最终采用方案Y解决”这样的结论性描述。

[0061] 第二个年龄层为历史层,对应待压缩区中处于中间位置的消息,占待压缩消息总量的中间30%。对历史层施加中等压缩强度,保留技术决策和文件修改记录,但省略完整的错误堆栈和逐步调试过程。例如,保留“修改了文件A的函数B以实现功能C”这类决策记录,但不保留完整的错误堆栈信息。

[0062] 第三个年龄层为近期层,对应待压缩区中较新的消息,占待压缩消息总量的后50%。对近期层施加低压缩强度,保留所有技术细节,包括代码片段、精确文件路径、完整错误信息和修改记录。近期层的消息与保留区的消息在时间上最为接近,其技术细节对于理解保留区内容的来龙去脉至关重要。

[0063] 根据上述三个年龄层的划分和各自的压缩强度要求,生成包含年龄差异化指令的压缩提示词,作为结构化语义压缩的输入。

[0064] 进一步地,在年龄差异化压缩指令生成之后,向压缩模型(可以是与主对话模型不同的、通常更轻量的语言模型)发送包含年龄指令和八段结构化模板的压缩提示词,要求模型按照优先级顺序生成结构化摘要。

[0065] 八段结构化模板依次为:第一段“初始请求与意图”(最高优先级),要求记录用户的原始请求、显式目标、约束条件和性能要求;第二段“关键技术概念”(高优先级),要求记录使用的技术栈及版本、架构模式、领域术语;第三段“文件与代码修改”(高优先级),要求记录涉及的完整文件路径、修改的函数/类、关键代码片段;第四段“错误与修复”(中优先级),要求记录遇到的错误信息、根因分析、解决方案、用户反馈修正;第五段“问题解决过程”(中优先级),要求记录尝试过的方案(包括成功和失败)、决策推理、权衡分析;第六段“用户消息记录”(中优先级),要求记录每条用户消息的精确内容、澄清和路线修正;第七段“待办任务”(较高优先级),要求记录未完成的任务清单、优先级、依赖关系;第八段“当前工作状态”(最高优先级),要求记录刚完成的工作、进行中的工作、精确的后续步骤、文件状态。

[0066] 压缩模型收到上述提示词后,按照年龄层指令对不同时段的消息施加相应的压缩力度,同时按照八段模板的结构组织输出,生成一份结构化的历史摘要。其中,第一段和第八段被赋予最高优先级,确保用户的原始意图和当前工作状态在任何压缩力度下都能完整保留。

[0067] 进一步地,语义压缩执行完成后,构建新的上下文结构,替换原有的对话历史。新的上下文结构由三部分组成:

第一部分为原始系统提示消息,即对话开始时设定的系统级指令,保持不变,不做任何修改。系统提示消息定义了智能体的角色、能力和行为规范,是智能体正确运作的基础。

[0068] 第二部分为压缩历史摘要消息,即压缩模型生成的结构化摘要。该消息被标记为[Compressed History],以明确区分其为压缩后的历史记录而非原始对话内容。摘要按八段结构化模板组织,包含了待压缩区中经过年龄感知差异化处理后的关键信息。

[0069] 第三部分为保留区的完整原始消息,即安全分割点之后的最近N轮消息。这些消息保持原文不做任何修改,确保智能体对最近对话的记忆完整无损。

[0070] 通过上述三部分的组合,重建后的上下文既保留了完整的系统指令和近期对话细节,又通过结构化摘要保存了历史对话中的关键信息,实现了在有限窗口内的信息最大化保留。

[0071] 进一步地,在所述第二层压缩处理执行后,还包括对压缩结果进行质量评估,包括:

基于摘要的结构完整性、信息密度、压缩效率及保留准确性四个维度计算加权综合质量分。

[0072] 具体地,每次第二层压缩执行完成后,对本次压缩的结果从四个维度进行量化评估,以监控压缩质量并为后续优化提供依据。

[0073] 第一个评估维度是结构完整性(权重30%)。检查压缩模型生成的摘要是否包含完整的八个结构化段落标记。若八个段落均存在,则该维度评分为100分;每缺少一个段落扣除相应分数。该维度确保八段式结构化框架的完整性,避免压缩模型遗漏重要信息类别。

[0074] 第二个评估维度是信息密度(权重25%)。计算摘要中每条消息的平均字符数,若平均字符数在100至500的范围内,则该维度评分为满分。低于100字符表明摘要过于简略,可能丢失重要信息;高于500字符表明压缩不够充分,浪费了窗口空间。该维度确保压缩率和信息保留之间的平衡。

[0075] 第三个评估维度是压缩效率(权重25%)。计算压缩比,即压缩后Token数与压缩前Token数的比值。若压缩比在30%至60%的范围内,则该维度评分为满分。低于30%表明压缩过度,可能造成信息损失;高于60%表明压缩效果不足,未能有效释放窗口空间。

[0076] 第四个评估维度是保留准确性(权重20%)。检查实际保留的消息数是否等于计算的预期保留轮数。若实际值等于预期值,则该维度评分为满分。该维度确保安全分割和上下文重建过程中消息数量的正确性。

[0077] 综合质量分通过以下加权公式计算:

综合质量分=结构完整性评分×0.30+信息密度评分×0.25+压缩效率评分×0.25+保留准确性评分×0.20;

综合质量分的取值范围为0至100分,可用于压缩质量监控、异常告警和策略优化。

[0078] 进一步地,每次第二层压缩执行完成后,除进行质量评估外,还对压缩的效能进行监测,以防止出现“压缩→回收极少→再触发→再回收极少”的无效循环。当对话进入晚期阶段,待压缩区的内容已被反复压缩到极致,新一轮压缩的Token回收量极低,此时继续触发LLM压缩不仅无法有效释放空间,还会浪费API调用成本。

[0079] 具体实现机制如下:

首先,计算Token回收量。每次压缩执行后,计算本次Token回收量,即压缩前上下文Token数减去压缩后上下文Token数。

[0080] 然后,更新连续低效计数器。若本次Token回收量低于最小回收阈值(如

500Token),则连续低效计数器加1;若回收量达到或超过最小回收阈值,则将连续低效计数器归零。

[0081] 接着,判断是否触发自动禁用。当连续低效次数达到预设上限(如3次)时,自动禁用第二层LLM压缩。禁用后,仅依赖Snip层进行轻量级维护,不再触发消耗API调用的LLM语义压缩。

[0082] 最后,设置手动恢复机制。当出现prompt-too-long等紧急情况(即上下文Token数已超出模型窗口限制导致API调用失败)时,可手动重置禁用状态,允许下一次压缩尝试,以应对突发的上下文膨胀。

[0083] 本实施例可应用于任何基于大语言模型的对话式智能体系统,包括但不限于编程助手、客服机器人、数据分析助手、文档处理助手等。本实施例具体阈值参数(如70%、92%、500Token等)可根据实际应用场景调整。第二层压缩可使用与主对话不同的(通常是更轻量的)语言模型执行,以降低压缩成本。

[0084] 本实施例还提供一种面向大语言模型智能体的多层级自适应上下文压缩系统,包括:

上下文监测模块,用于持续监测当前智能体对话上下文的Token使用率;

第一层压缩模块,用于当所述Token使用率超过第一预设阈值时,则执行第一层压缩处理,所述第一层压缩处理包括识别并裁剪未被当前上下文引用的工具结果消息内容;

第二层压缩模块,用于当所述Token使用率超过第二预设阈值时,执行第二层压缩处理,将新构建的上下文架构替换原有的历史对话;其中,所述第二预设阈值大于所述第一预设阈值;

反循环保护模块,用于监测每次第二层压缩处理执行所产生的Token回收量,当连续N次压缩的Token回收量均低于最小回收阈值时,则自动禁用所述第二层压缩。

[0085] 其中,本实施中的各层压缩模块可独立部署,也可组合使用。

[0086] 为了更加清楚的表达本发明的技术方案,下面提供具体实施例进行方案介绍:

实施例1、简单客服问答场景:

某企业客服智能体处理用户咨询,对话窗口为100K Token:

1.对话进行到第50轮,Token使用率达72%→Snip层启动,裁剪了15条早期工具调用结果(知识库检索结果),Token使用率降至61%。

[0087] 2.继续对话至第80轮,使用率达93%→LLM压缩启动。

[0088] 自适应算法判断:消息长度短(0分)、无工具使用(0分)、无错误(0分)、无代码(0分),总分0,保留5轮。执行压缩后使用率降至42%。

[0089] 3.智能体无感知地继续为用户提供服务,对话可持续进行。

[0090] 实施例2、复杂代码重构场景:

某开发者使用编程智能体进行大型代码库重构,对话窗口为200K Token:

1.第30轮,使用率75%→Snip层裁剪了22条过时的文件读取结果,使用率降至58%。

[0091] 2.第45轮,使用率94%→LLM压缩启动。

[0092] 自适应算法判断:消息长度2500字符(2分)、工具调用18次(2分)、有错误堆栈(1分)、代码占比60%(2分),总分7,保留10轮。

[0093] 年龄分层压缩:远古层高度压缩保留架构决策,近期层保留完整代码修改记录。

[0094] 3.压缩质量评分89/100,压缩比38%,使用率降至43%。

[0095] 4.第65轮,再次达到92%,执行第二次压缩。前一次摘要被进一步压缩整合。

[0096] 5.第70轮起,连续3次压缩回收量均低于500 Token→反循环保护启动,禁用LLM压缩,仅依赖Snip层维护。

[0097] 以上,仅为本申请较佳的具体实施方式,但本申请的保护范围并不局限于此,任何熟悉本技术领域的技术人员在本申请揭露的技术范围内,可轻易想到的变化或替换,都应涵盖在本申请的保护范围之内。因此,本申请的保护范围应该以权利要求的保护范围为准。

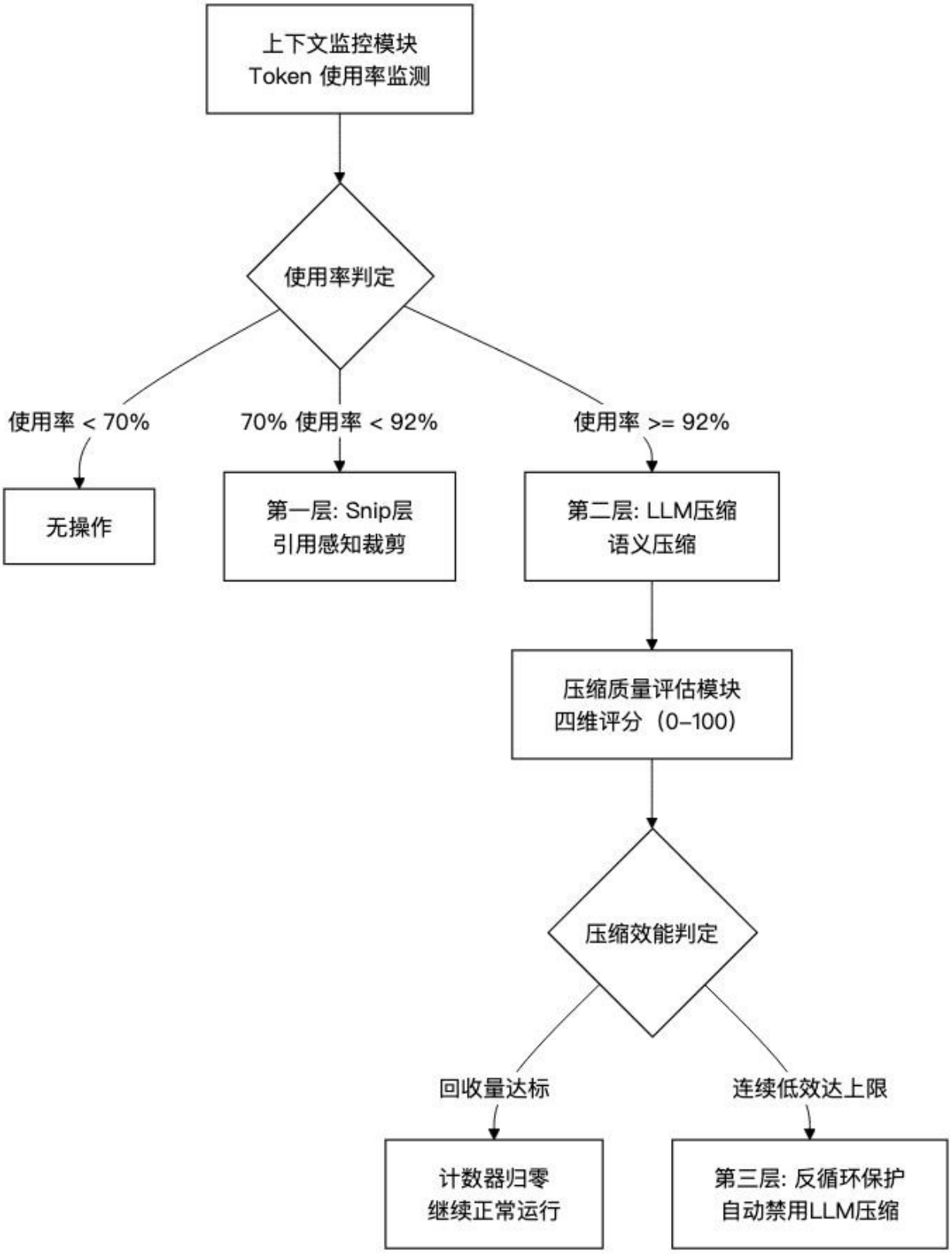

上下文监控模块
Token 使用率监测
使用率判定
使用率 < 70%
70% 使用率 < 92%
使用率 >= 92%
无操作
第一层: Snip层
引用感知裁剪
第二层: LLM压缩
语义压缩
压缩质量评估模块
四维评分（0-100）
压缩效能判定
回收量达标
连续低效达上限
计数器归零
继续正常运行
第三层: 反循环保护
自动禁用LLM压缩

图1

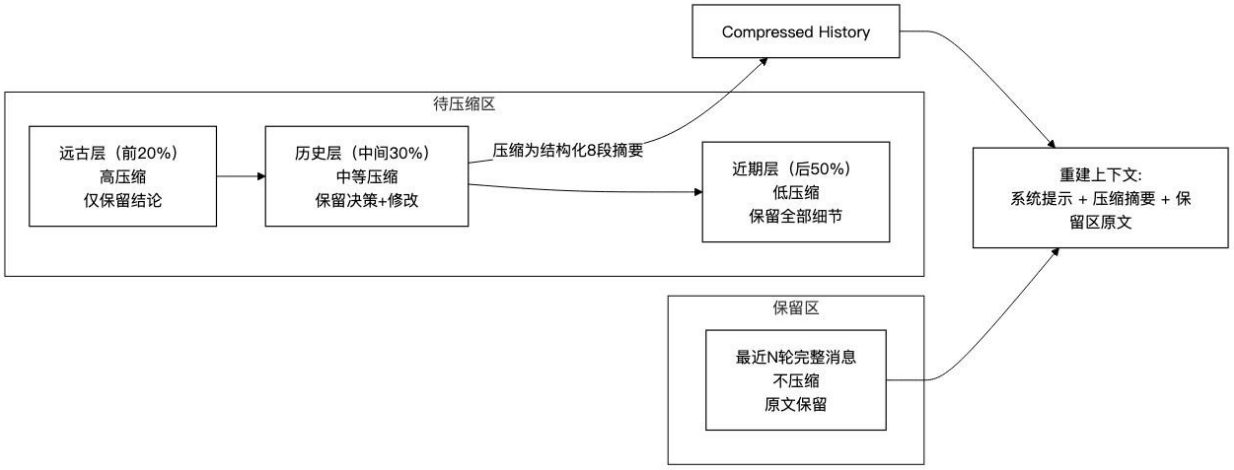

Compressed History
待压缩区
远古层（前20%）
高压缩
仅保留结论
历史层（中间30%）
中等压缩
保留决策+修改
压缩为结构化8段摘要
近期层（后50%）
低压缩
保留全部细节
重建上下文:
系统提示 + 压缩摘要 + 保留区原文
保留区
最近N轮完整消息
不压缩
原文保留

图2

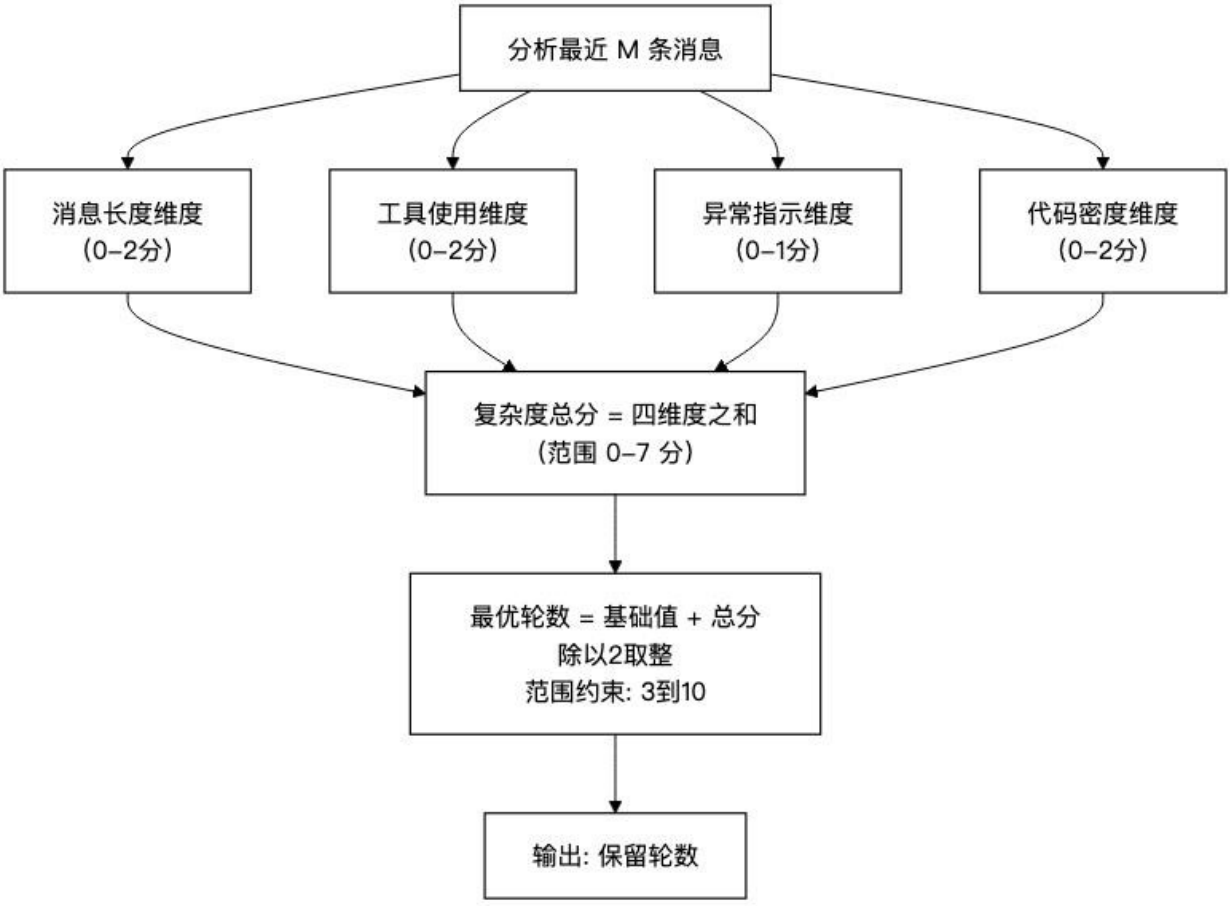

分析最近 M 条消息
消息长度维度
(0–2分)
工具使用维度
(0–2分)
异常指示维度
(0–1分)
代码密度维度
(0–2分)
复杂度总分 = 四维度之和
(范围 0–7 分)
最优轮数 = 基础值 + 总分
除以2取整
范围约束: 3到10
输出: 保留轮数

图3

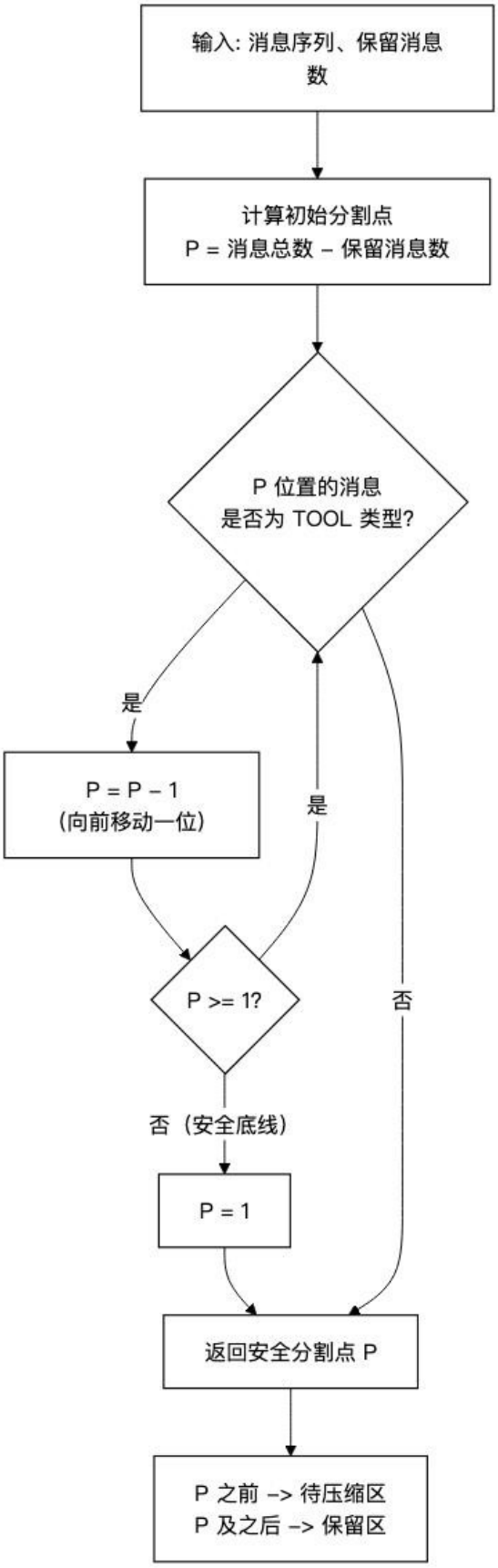

输入: 消息序列、保留消息数
计算初始分割点
P = 消息总数 – 保留消息数
P 位置的消息
是否为 TOOL 类型?
是
P = P – 1
(向前移动一位)
是
P >= 1?
否
否 (安全底线)
P = 1
返回安全分割点 P
P 之前 -> 待压缩区
P 及之后 -> 保留区

图4